03. Introduction to Binary Decision Diagram (BDD)

❗ **Copyright and Ownership Notice**

關於這個 notion page 以及其包含的所有 pages （以下統稱「本 page」），其著作權與所有權是屬於台大電機系黃鐘揚教授以及其所開設的系統晶片驗證 (SoC Verification, SoCV) 課程之所有助教 (以下統稱「作者們」) 所擁有。

基於推廣知識技術與服務大眾之精神，本 page 採取 MIT 授權，即任何人皆可因任何目的使用、複製、修改、發布、與再散佈本 page，包括商業用途。不需簽署協議，也不需支付授權費，惟任何的使用與散佈必須包含本著作權與所有權聲明。如有違反本著作權與所有權聲明，本 page 之作者們仍保有相關的法律追朔權利。詳細關於 MIT 授權的規範請見 OSI 官網。

本 page 之內容以「現狀」提供 (provided "as-is")，不附帶任何保證。對於本 page 之內容如有任何疑問或是建議，請來信 ntueesocv@googlegroups.com.

The copyright and ownership of this Notion page and all pages it contains (hereinafter referred to as "this page") belongs to Professor Chung-Yang Huang of Department of Electrical Engineering, National Taiwan University, and all teaching assistants of this SoC Verification (SoCV) course that he teaches.

In the spirit of promoting knowledge, technology and serving the public, this page adopts the MIT License, meaning anyone can use, copy, modify, publish, and redistribute this page for any purpose, including commercial use. No agreement needs to be signed, and no licensing fee needs to be paid, provided that any distribution must include this copyright and ownership notice. In case of violation of this copyright and ownership notice, the authors of this page still retain the relevant legal right to take retroactive action. For detailed regulations regarding the MIT License, please see the OSI official website.

The content of this page is provided "as-is", without any warranty. If you have any questions or suggestions regarding the content of this page, please email ntueesocv@googlegroups.com.

▼ Key Learnings in this Lecture Note

+ Review on property checking and explicit model checking techniques

+ BDD — an efficient and canonical data representation for Boolean functions

+ How to construct BDDs for a Boolean function/circuit

+ Play with the BDD calculator in GV

▼ Previous Lecture Notes

attachment:160758a6-1043-4cd0-b39a-41da29e42077:03._Introduction_to_BDD.pdf

▼ Recall: Formally Verifying a Property

▼ "Mathematical Certainty" in Formal Verification

▼ Space exhaustiveness

- Verify all input combinations of the system

▼ Time exhaustiveness

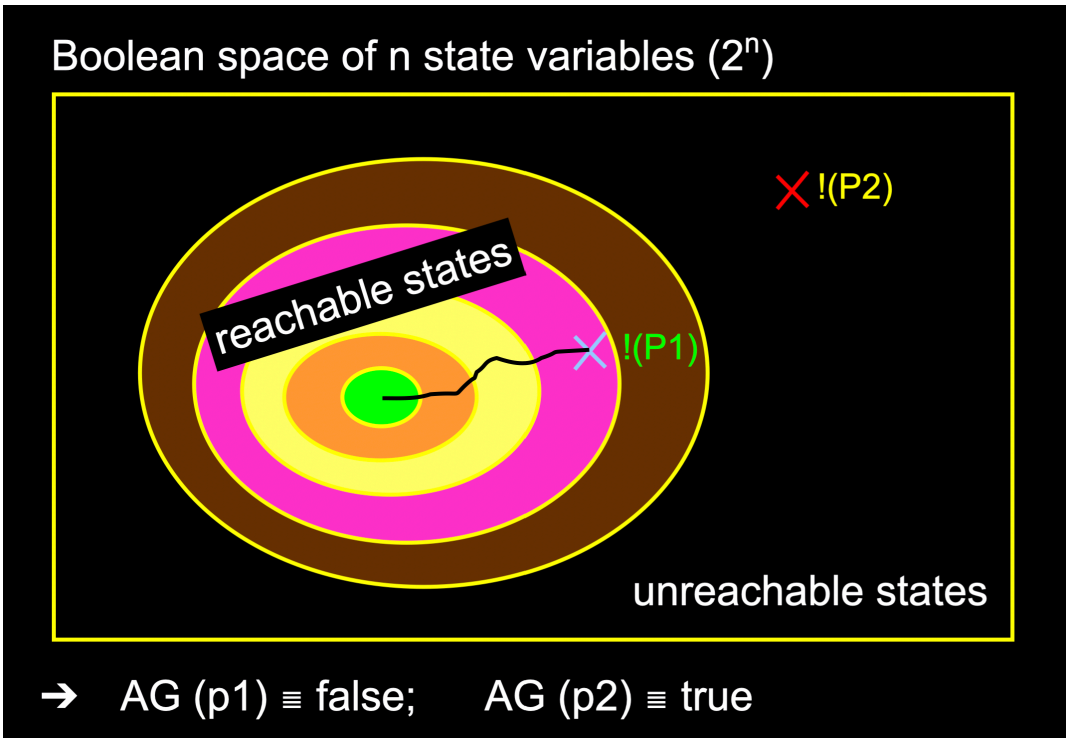
- Verify system behavior from initial state to time infinity

attachment:c2259c98-5b22-433e-8074-cac6f9df2bd9:math_certainty.mov

▼ Set of Reachable States

attachment:2deffafb-894e-42d8-9a5c-e0d86e4598d9:set_of_reachable_states.mov

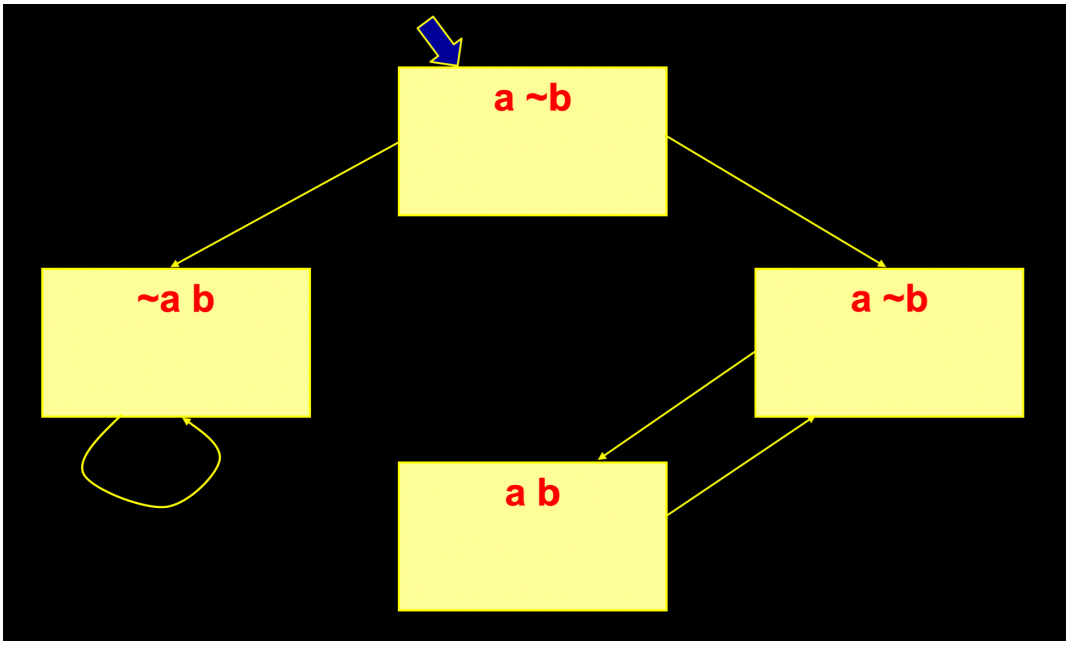
▼ Boolean State Space



▼ Explicit model checking algorithm

```
For an initial state s0 and any state s in the model
M, check M, s ⊨ f
```

▼ Example: M, s ⊨ ¬AF ¬a ∧ AF b ?



▼ Algorithm

1. Find the atomic formulae in f
   - i.e. { AF ¬a, AF b }
2. For each state s in M, label the truthfulness of these atomic formulae
   - Usually based on DFS or BFS order.

   attachment:23acc9be-725d-479f-bb8a-134b8338faf9:Explicit_Model_Checking.mov

3. Compute the truthfulness of f with the given initial state s0
   - AF b && !AF!a
     ⇒ f is true!

😨 ▼ **Complexity Issues**

- Given any meaningful size of a design, we won't be able to formally verify the properties with the "explicit" model checking algorithm.
- The number of states is exponential to the number of registers (FFs). How can we handle the complexity if the explicit enumeration is impossible?

⇒ **Efficient data structure to represent the set of reachable states**
and an arbitrary logic function?

▼ Introduction to BDD — An Efficient and Canonical Data Structure to Represent Boolean Functions

- See pp. 4 ~ 19 of the previous lecture note

▼ (Manually) Constructing BDDs for a Boolean Function/Circuit

See pp. 20 ~ 33 of the previous lecture note

▼ Efficient BDD Algorithm Implementation

See pp. 34 ~ 52 of the previous lecture note

▼ BDD Calculator in GV

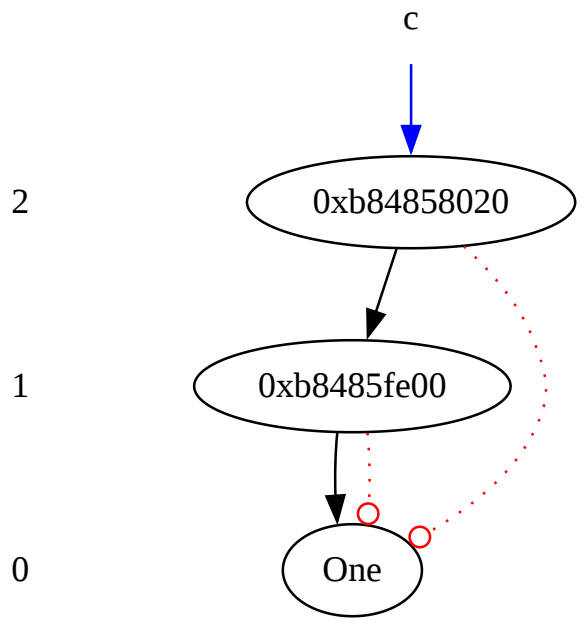
💪 **GV includes a simple and yet efficient BDD implementation in C++**

https://github.com/ric2k1/RicBDD

▼ A simple example

```
SETUP> bsetv 1 a     // associate var 'a' with level 1
(the lowest)
SETUP> brep a
[1](+)
  [0](+)             // level-0 node => CONST 1
  [0](-)  (*)        // (-) means neg-edge

==> Total #BddNodeVs : 2

SETUP> bsetv 2 b
SETUP> band c a b    // c = a && b
SETUP> brep c
[2](+)
  [1](+)
    [0](+)
    [0](-)  (*)
  [0](-)  (*)

==> Total #BddNodeVs : 3

SETUP> bdraw c c.dot
```

▼ Visualized by “Graphviz Interactive Preview” vs-code extension



▼ [LN] BDDs for different Boolean functions

- Try to build BDDs from different types of functions and see how they look like. For example:
  - Adder
  - Multiplier
  - Counter
  - Random logic function
  - Anything else?
- Discuss the growth of the number of BDD nodes w.s.t. input sizes. What's the complexity?

[Hint] Instead of writing lengthy dofiles for the BDD calculator, you are encouraged to write a program to convert a simple function to BDDs. You can refer to "testBdd.cpp" in https://github.com/ric2k1/RicBDD about how to build BDDs in a C++ program.

▼ [LN] The worst-case size of BDDs

- The worst-case size of BDDs is known to be exponential in terms of the number of variables of the Boolean functions. Before reading the referenced paper below, how do you compute/estimate the worst-case size of BDDs for any arbitrary functions?
- Please refer to the discussion in this paper: https://hal.archives-ouvertes.fr/hal-01880774/document. Summarize the paper: what is the worst-case BDD size and how to derive it. Please highlight some interesting points from the paper
- Design some experiments to verify the statements of this paper